

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00169348 B00175178
Student Name:	Alya Khalil Hanein Atalla

Module Information

Module Title:	Scripting for cybersecurity
Module Code:	Comp7207

Assessment Information

Assessment Title:	Group project
-------------------	---------------

Declaration

☐ YES I confirm that I have read and understood the University's Academic Integrity Policy.

<https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/>

☐ YES I have completed the TU Dublin AI Driving License theory test.

<https://www.tudublin.ie/explore/about-the-university/academic-affairs/ita/ita-resources/artificial-intelligence-driving-license/>

☐ YES I declare that this assessment submission is my own work.

☐ YES I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	
Date:	

Introduction

This project mainly focuses on the implementation and the design of a realistic cybersecurity automation tool that uses both the powershell and python to analyze any activity but also gives the result of external threat intelligence. The main aim of this project was to show how we could use scripting as an automation tool when it comes to security tasks that more than likely would need multiple long steps

The tools to implement this were used to identify an active network connections that would be on a host system and to extract any external ip addresses that's involved in these connections. The ip addresses are also checked against abuseIPDB threat intelligence service that are checked if they have been reported previously for malicious attacks. By automation with this sort of workflow the tools lets us check information gotten by a network without having to do a lot of searching manually.

This project was made with scripting and powershell was used for any system level data that needed to be collected. Python was also used to check the collected data and then interact with an API at a security level. The output thats given is a JSON file that gives a summary of any external IP addresses that have been detected. This project also shown a lot of things such as, scripting skills, API integration and the application of automation in a real world cybersecurity scenario.

Tool design and implementation

The tool used was a two stage workflow, each stage that was tested out was implemented by using a scripting language useful for what the task was. Powershell was used for system collection and python was used for data processing, and threat intelligence.

Powershell component

The powershell part is responsible for collecting the live network from the host system. We used powershell for this task because it has strong capabilities when it comes to system administration and its ability to work with something structured instead of raw text. These features make powershell very suitable for suitable forensic and administrative scripting when it comes to cybersecurity.

The scripts created help perform network configuration by making a lot of established TCP connections in the system. It performs a network command and it also filters the output to include only important connections that are established. The filtering that is done is very important as established connections represent communication that is active when it comes to the host and remote system.

powershell pipelines are used to process command output at a high standard. The data that has been filtered is then converted into a custom powershell object. This lets things such as remote address, local address and the connection state to be seen. The objects are then sorted out to improve readability and the consistency of the output.

The script is then implemented using support parameters and functions. The final output that is given is a CSV file. We chose CSV because it was a portable format that is easily readable by other tools and python

Python component

The python component is for improving the network data that was collected with external threat intelligence. Python was used because of the amount of libraries it has and the language we were most familiar with making it good when working with APIs and creating structured data such as JSON

The script reads the configured data from the JSON file and the configurations in the file had the API key that was needed to authenticate the abuseIPDB service. As we stored data separately from the source code it helped us improve security and flexibility the script read the CSV file that was generated by PowerShell Using the built in file of input and output the python data structures let the script to look through each network connection entry

Every ip address that was external let the script to send a request to the abuseIPDB API using the requests library. The API response gave us that the key information such as the country it was associated with and abuse confidence is taken, error handling was something that we implemented using 'try except' blocks that made sure any sort of network issues or API failures don't let the scripts that have been created to crash

The results that was given are stored in a list of dictionaries and are written to a structured JSON output file. The report that is made by JSON provides a summary of the reputation external ip addresses and it also shows the last output of automated workflow.

Cybersecurity context

The task given by this project reflects a realistic cybersecurity scenario that is usually encountered in security operations and incident response. Analyzing a network connection is an important activity when doing an investigation on potential security incidents such as unauthorized access, malware infections and data exfiltration attempts. External ip addresses that are involved within the network communicate benign services but also can indicate malicious infrastructure which include command and control servers, scanning hosts or known attacker systems.

Threat intelligence is a huge and important role in helping analysts make an informed decision when it comes to investigation. Services such as AbuseIPDB check community reported data about malicious ip addresses which provide an insight into whether an Ip has been associated with any malicious or abusive behavior. By correlating the local system observations with external threat intelligence an analysts can put alerts at a higher level and prioritize it, they can then focus their efforts on the suspicious activity they come across.

When automating this process it provides significant benefits when it comes to operating. Instead of extracting IP addresses, searching for reputation information, compiling reports manually, an analyst can rely on a scripting workflow to perform steps that are consistent and quick. This then reduces investigation time and also allows security teams to respond faster to potential threats.

This tool mirrors real world workflow by checking a system level data collection with enhancement of threat intelligence. This demonstrated a lot of principles that scale to larger security automation platforms used in an enterprise environment

Tool demonstration

This tool that is executed is a two step process that reflects a combined workflow between powershell and python

The first thing is when the powershell script is executed using powershell core. When this is ran the script establishes network connections on the host system and then processes the output using the sorting, filtering and custom objects. The data that is given is exported to a CSV file that we named `network_connections.csv`. This file has relevant network information that's needed to be analyzed further.

Then, the python script is executed it reads the configuration file so it can get the API key and then it parses the CSV file that has been generated by the powershell script. For any external IP address that is found and identified the script sees the abuseIPDB API and retrieves data that is reputational. If any errors did occur during an API request it's handled properly

Lastly, the script writes to a json file called `final_report.json` that puts the results we got in it. This file gives us the final output and gives us a clear summary of the threat intelligence that was found. The output that was successful showed us the combined workflow operates correctly from the data that has been collected.

Testing and validation

The testing and validation has been carried out to make sure that both the tool functioned properly on its own when it was combined with a single workflow. Testing focused mainly on verifying data properly, handling the cases and ensuring a reliable interaction with external threat intelligence services.

The powershell part was tested by executing the script within the host system under normal network conditions. The output was checked and validated by manually comparing a subset of the csv data that was exported with the result of the network inspection and commands to confirm that only an established TCP connection would be captured. Additional checks were done to make sure that local and loopback addresses were correctly included in the raw output so that filtering could be applied in the python stage later on. Sorting and creation of objects were verified by reviewing the structure and consistency of the CSV file that was generated.

When the python part was tested it used multiple kinds of scenarios. When we began at the start the testing was performed using a benign external IP address to make sure that API requests were sent properly and that the responses were given correct. The testing that was done later involved IP addresses that existed on abuseIPDB to make

sure that it was relevant because the abuse confidence score and the country were taken and stored good. The script that was made also was tested with small files and empty ones to make sure it was able to handle any sort of cases where no external IP addresses were present without it failing.

error handling was tested using an API key that we both knew wouldn't work to simulate network and connectivity issues. In this scenario the script handled everythingn very good and kept executing it without it crashing which made us confirm the tool would work when put to the test.

as we reach the end of our testing, end to end testing was performed by running everything we had in our workflow from the data collection within our powershell to python enhancement and json report generation. the json report being a success confirmed to us that the correct data flow between the components and proper operation of the integrated workflow.

Reflection

This project was completed as a group assignment by Alya and Hanein. With the end design, testing and documentation we both carried it out together as it is a group project. Some responsibilities were taken by each other but it was reviewed and refined by us both

Alya focused mainly on the powershell component of the project. This needed a lot of research on what suitable methods could help enumerate active network connections, implementing filtering, creating a custom object for a structured output and also sorting when using pipelines. I was responsible for making sure the powershell script was collecting accurate data when it came to network connection and then exported in a CSV format that could be used by python. When I ran into issues I consulted Henein and we worked together to fix things properly to make sure it was okay.

Henein focused mainly on developing the python component and integrating the abuseIPDB threat intelligence service. This also involved parsing configuration data from a JSON file, making sure API could authenticate properly, sending requests to the abuseIPDB API but doing this with an external library and also being able to process API responses in a structured JSON report. I also handled many challenges when it came to the python script running correctly on macOS and any API issues such as responses and empty result sets.

After completing both components, we both worked together to integrate PowerShell and the python scripts in a single workflow. Both of our joint effort was needed when debugging and testing especially when seeing the data flow within the scripts to make sure a consistent output. With us working together we were able to understand both components and understand how we can use scripting language in a realistic cybersecurity task. This project showed both of us how teamwork is important and if we work together we can fix many issues, using communication throughout the entire project.